

# Chronos

## A Content-Addressed Versioned Data Store

Final architecture, Dolt positioning, research methodology, frontend workflow, and one-week delivery plan

DECISION

**Build the final platform around a Go-based persistent Prolly Tree. Preserve the Python Merkle trie as the correctness oracle, benchmark baseline, and emergency fallback.**

| Project statement                                                                     | Delivery constraint                                                              | Primary differentiator                                  |
|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|---------------------------------------------------------|
| Git-inspired structured-data versioning through content-addressed Merkle search trees | One week to a frontend-ready final platform, paper, benchmarks, and presentation | Independently verifiable diff proofs over version roots |

Prepared for project-partner alignment

19 August 2026

## 01 EXECUTIVE ALIGNMENT

# What we are building - and what we are not

**ONE-SENTENCE GOAL**

**Chronos will be a focused, persistent structured-data versioning platform that stores immutable content-addressed chunks, supports Git-like history operations, and can prove a reported diff against two trusted root hashes.**

## Why the direction changed

The earlier Python implementation successfully validated structural sharing and hash-pruned diff. This is now the final submission, so the deliverable must present a coherent system rather than only an algorithm demonstration. The final backend therefore moves to Go and an ordered Prolly Tree, while retaining Python as a reference implementation.

## Final system promise

- A user can import structured data, create versions, inspect history, compare any two versions, and restore an earlier state.
- A small update rewrites only a local chunk region and its ancestor path; unchanged chunks are reused by hash.
- Data persists across process restarts and referenced chunks are integrity-checked when read.
- The system can produce a compact proof that a claimed set of changes is consistent with two version roots.
- Every headline claim is backed by a reproducible workload, raw results, and clearly stated limitations.

## Non-goals for this week

| Do not build                         | Reason                                                                         |
|--------------------------------------|--------------------------------------------------------------------------------|
| SQL parser or MySQL compatibility    | Would turn a versioning project into a query-engine project.                   |
| Production multi-writer transactions | Requires concurrency control, durable logging, and a much larger test surface. |
| Three-way merge                      | Useful but conflict semantics would consume the schedule.                      |
| Distributed push/pull replication    | Not required to demonstrate local versioning or proof verification.            |
| Authentication and tenancy           | Deployment concern, not the research contribution.                             |
| Automatic garbage collection         | Safe reachability-based reclamation can remain future work.                    |

## 02 STATE OF THE ART

# What Dolt does and where Chronos fits

Dolt is the correct state-of-the-art reference, but it is not a fair all-purpose opponent for a one-week academic datastore.

## Dolt in practical terms

Dolt is a production, MySQL-compatible relational database with Git-like version control. It stores table data and indexes in ordered, history-independent Prolly Trees inside a content-addressed Merkle DAG. It supports commits, branches, diff, schema and row merge, clone, push, pull, transactions, persistence, and garbage collection.

| Dimension          | Dolt                                                              | Chronos final target                                        |
|--------------------|-------------------------------------------------------------------|-------------------------------------------------------------|
| Data model         | Relational SQL tables, schemas and indexes                        | Structured key-value / JSON rows with CSV workflow          |
| Tree               | Production Prolly Trees, about 4 KiB average chunks               | Compact educational Prolly Tree with deterministic chunking |
| Persistence        | Append-only chunk store plus production lifecycle                 | Persistent content-addressed chunks and atomic refs         |
| Version control    | Commit, branch, diff, merge, clone, push and pull                 | Commit, branch pointer, diff, log and checkout              |
| Research extension | Fast authenticated structures; no documented diff-proof interface | Explicit diff-proof generation and independent verification |
| Maturity           | Production database                                               | Focused final-year systems artifact                         |

## Claims we must not make

- Chronos invented content-addressed database versioning, structural sharing, or hash-pruned diff.
- Chronos is a replacement for Dolt or a production SQL database.
- A faster in-process operation means Chronos is faster than Dolt overall.
- Internal units such as nodes compared are directly comparable across different engines.

## Defensible positioning

Chronos is a compact research system for studying Merkle search-tree trade-offs in structured-data versioning. Dolt establishes that Prolly Trees work at production scale; Chronos isolates the mechanics, exposes detailed instrumentation, compares design choices, and adds verifiable diff proofs as a focused extension.

## 03 TARGET ARCHITECTURE

# The final platform at a glance

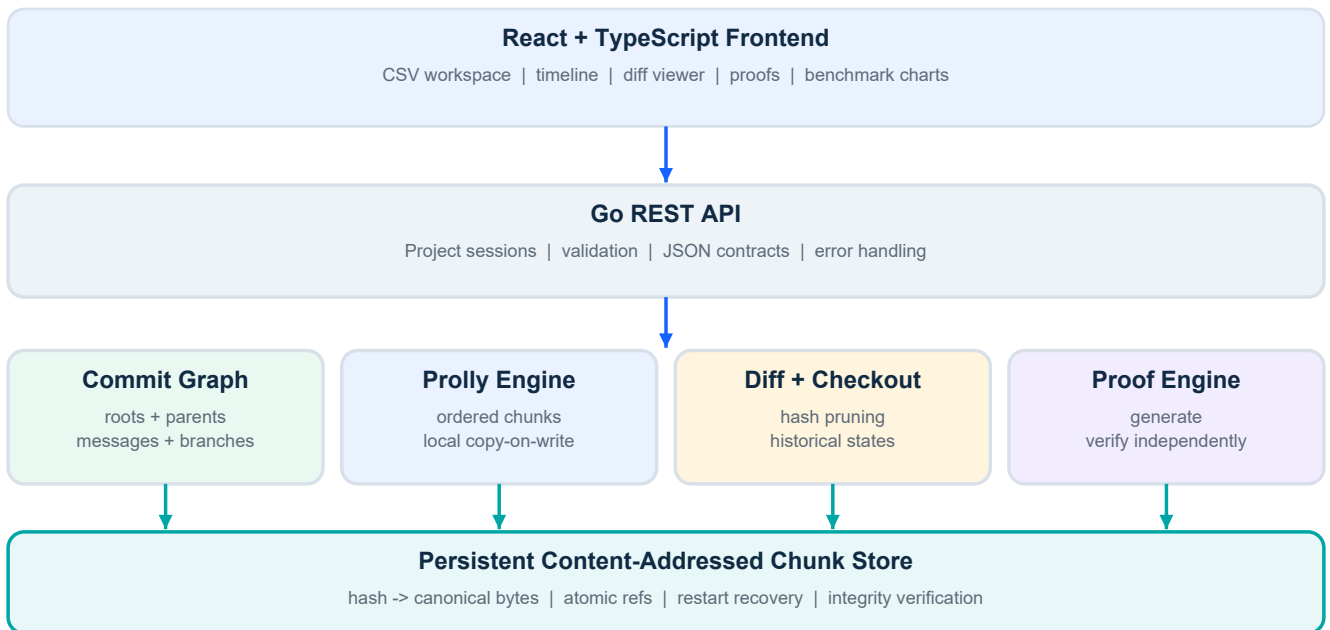


Figure 1. Proposed final Chronos platform architecture.

## Component responsibilities

| Component         | Responsibility                                                            | Acceptance signal                         |
|-------------------|---------------------------------------------------------------------------|-------------------------------------------|
| React frontend    | Import, history, diff, checkout, proof and results experiences            | Complete demo without terminal commands   |
| Go API            | Stable JSON contracts, validation, project lifecycle and error responses  | API integration tests pass                |
| Commit graph      | Parents, roots, messages, timestamps and named branch pointers            | History survives restart                  |
| Prolly engine     | Ordered chunks, deterministic boundaries and local copy-on-write mutation | Incremental root equals full rebuild root |
| Diff/proof engine | Hash-pruned comparison plus authenticated evidence                        | Tampering causes verification failure     |
| Chunk store       | Store canonical bytes by hash and publish refs atomically                 | Restart and integrity tests pass          |

04 ARCHITECTURE DECISION

# Why Go + Prolly Tree is the final target

| Decision factor       | Python fixed Merkle trie                       | Go Prolly Tree                               |
|-----------------------|------------------------------------------------|----------------------------------------------|
| Current readiness     | Existing tested prototype                      | Requires focused implementation              |
| Key order             | Destroyed by hash routing                      | Preserved                                    |
| Range access          | Requires materialize and sort                  | Native ordered traversal                     |
| Chunk behavior        | Fixed depth; leaves become wider as data grows | Content-defined, scale-stable average chunks |
| History independence  | Deterministic for a full state                 | Deterministic across insertion histories     |
| Deployment            | Python runtime and application packaging       | Single backend binary                        |
| Role in final project | Oracle, baseline, fallback                     | Primary backend if gate passes               |

THE RULE

**Do not delete or rewrite the Python implementation. Run the Go engine beside it and use Python to validate roots, diffs, and benchmark workload correctness.**

## Twenty-four-hour implementation gate

The Go path continues only if all six conditions pass within the first day:

| # | Gate                     | Pass condition                                             |
|---|--------------------------|------------------------------------------------------------|
| 1 | Deterministic bulk build | Same sorted state always yields the same root.             |
| 2 | Local mutation           | One update avoids rebuilding the complete leaf list.       |
| 3 | Correct ordered access   | Point get and bounded range scan match a sorted reference. |
| 4 | History independence     | Random insertion orders converge on the bulk-build root.   |
| 5 | Persistence              | Close, restart, reopen and recover commits and roots.      |
| 6 | Property testing         | Random put/delete sequences match a full rebuild oracle.   |

Fallback trigger: if local chunk-boundary propagation remains incorrect after the gate, ship the tested Python incremental trie as the backend and retain the Go Prolly work as an evaluated prototype. The frontend contract must remain unchanged.

05 STORAGE MECHANICS

# How an incremental commit works

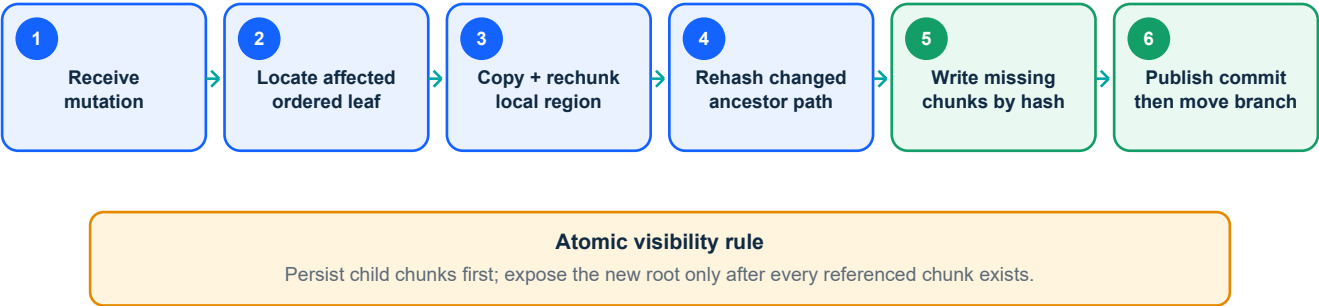


Figure 2. Copy-on-write mutation and atomic commit publication.

## Required invariants

| Invariant         | Meaning                                                         | Test                                                |
|-------------------|-----------------------------------------------------------------|-----------------------------------------------------|
| Content identity  | A chunk hash is computed only from canonical chunk bytes.       | Rehash every loaded chunk and reject mismatch.      |
| Immutability      | Published chunks are never modified in place.                   | Old versions remain unchanged after later commits.  |
| Root equivalence  | Incremental mutation yields exactly the root of a full rebuild. | Property tests over randomized workloads.           |
| Reachability      | Every published root references only present child chunks.      | Walk all references after every commit and restart. |
| Atomic visibility | A commit appears only after every referenced chunk is durable.  | Fault injection before and after ref publication.   |
| History binding   | Commit identity includes root, parent and metadata.             | Changing a message or parent changes the commit ID. |

## Suggested persistent layout

| Path / record | Contents                                                           |
|---------------|--------------------------------------------------------------------|
| chunks/       | Canonical encoded leaf or internal chunk; write-if-absent.         |
| commits/      | Root, parent, timestamp, message and optional provenance metadata. |
| refs/heads/   | Atomic pointer to a commit ID.                                     |
| project.json  | Format version, active branch, dataset schema hints and settings.  |

## 06 RESEARCH CONTRIBUTION

# Verifiable diff proofs

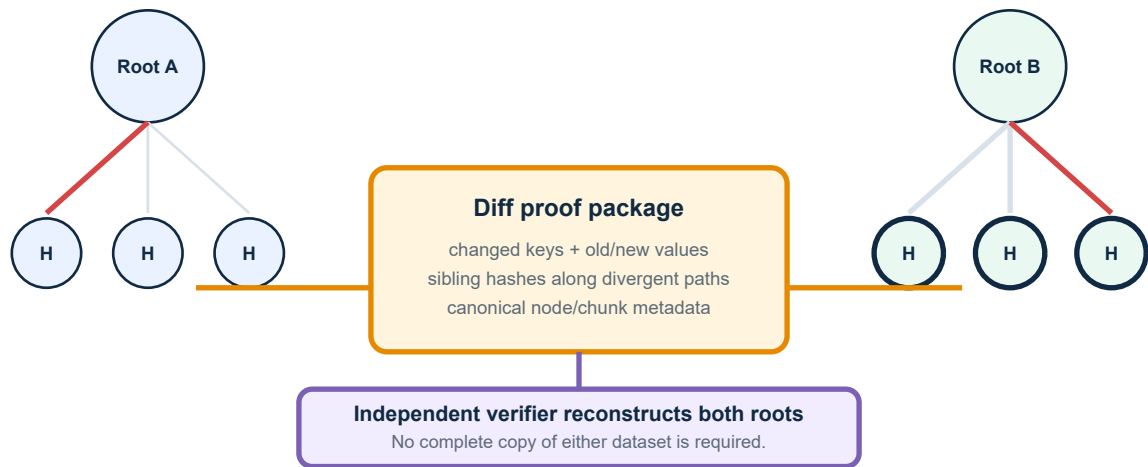


Figure 3. A proof carries changed values and authentication paths sufficient to reconstruct both roots.

## Problem

A normal diff tells a user what the server claims changed. A verifier that trusts only Root A and Root B would ordinarily need both complete trees to recompute that result. Chronos should instead export a compact evidence package for only the divergent paths.

## Proof contents

- Changed keys with their old/new values or explicit absence markers.
- Sibling hashes needed to reconstruct each affected path under both roots.
- Chunk-boundary and ordering metadata required for deterministic reconstruction.
- The canonical format version and hash algorithm identifier.

## Verification obligations

| Check                             | Failure that must be detected                                       |
|-----------------------------------|---------------------------------------------------------------------|
| Root reconstruction               | A forged value, sibling hash, or path.                              |
| Ordering and boundary rules       | A proof that omits or relocates keys improperly.                    |
| Completeness of divergent regions | An undisclosed changed path.                                        |
| Canonical encoding                | Alternative bytes that ambiguously represent the same logical node. |
| Root pair binding                 | Reusing a valid proof for different versions.                       |

## Evaluation hypothesis

*For sparse changes, proof generation and verification should grow primarily with the number of changed paths and tree depth, while proof size should remain far smaller than transferring both complete datasets.*

07 FINAL USER EXPERIENCE

# The complete frontend workflow

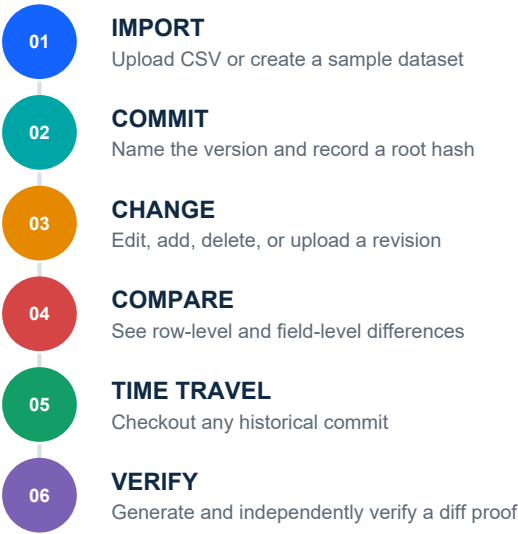


Figure 4. The six-step demonstration flow the partner should be able to present without a terminal.

## Essential screens

| Screen         | Must show                                                   | Primary action          |
|----------------|-------------------------------------------------------------|-------------------------|
| Project home   | Recent project, active branch, dataset size and latest root | Create/open project     |
| Import         | CSV preview, key-column selection and validation errors     | Create initial commit   |
| Data workspace | Searchable rows and pending changes                         | Edit/add/delete row     |
| Commit history | Graph/timeline, message, parent, timestamp and root         | Select two commits      |
| Diff viewer    | Added, removed and field-level modified records             | Generate proof          |
| Checkout       | Read-only historical state and clear version banner         | Return to active branch |
| Verify         | Root pair, proof summary, pass/fail result and tamper demo  | Verify proof            |
| Evaluation     | Charts, workload controls and downloadable raw results      | Run or load benchmark   |

## Presentation principle

Lead with the versioning experience, not the data structure. Show the root and structural statistics as evidence after the user sees a useful operation. The proof-verification failure after one tampered value should be the memorable final moment.

## 08 ENGINEERING WORK

# Everything that has to be implemented

| Work package              | Required deliverables                                                     | Done when                                          |
|---------------------------|---------------------------------------------------------------------------|----------------------------------------------------|
| Repository reconciliation | Preserve local edits; integrate six remote commits; rerun tests           | One clean base and documented branch strategy      |
| Go project skeleton       | Packages for chunks, tree, commits, proofs and HTTP                       | Build and test commands work from a fresh checkout |
| Canonical encoding        | Stable leaf/internal/commit bytes and format version                      | Golden fixtures reproduce hashes                   |
| Prolly bulk build         | Sorted input, content-defined leaf/internal chunking                      | Deterministic roots across runs                    |
| Local mutation cursor     | Get, range scan, put and delete with boundary propagation                 | Incremental roots match rebuild oracle             |
| Persistent store          | Write-if-absent chunks, atomic refs, restart loading and integrity checks | Fault/restart tests pass                           |
| Version graph             | Commit, log, checkout and lightweight named branches                      | History navigation works after restart             |
| Diff                      | Typed added/removed/updated rows with field changes                       | Matches brute-force comparison                     |
| Proofs                    | Generate, serialize and independently verify                              | All tamper cases fail safely                       |
| HTTP API                  | Projects, imports, rows, commits, history, diff, checkout, proofs, stats  | Contract tests cover happy and error paths         |
| Frontend                  | All eight essential screens and responsive navigation                     | Full demo needs no terminal                        |
| Benchmarks                | Chronos engines, baselines and pinned Dolt adapter                        | Raw CSV plus one-command reproduction              |
| Paper + presentation      | Method, results, limitations, architecture and rehearsed demo             | Every claim traces to evidence                     |

## 09 INTEGRATION CONTRACT

# Backend endpoints and shared definitions

## Minimum API

| Method | Endpoint                             | Purpose                                         |
|--------|--------------------------------------|-------------------------------------------------|
| POST   | /api/projects                        | Create a persisted Chronos project.             |
| POST   | /api/projects/{id}/import            | Import CSV and create the first commit.         |
| GET    | /api/projects/{id}/rows              | Read active or historical rows with pagination. |
| PATCH  | /api/projects/{id}/rows/{key}        | Stage a structured row update.                  |
| DELETE | /api/projects/{id}/rows/{key}        | Stage a row deletion.                           |
| POST   | /api/projects/{id}/commits           | Publish staged changes with a message.          |
| GET    | /api/projects/{id}/commits           | Return history and branch pointers.             |
| GET    | /api/projects/{id}/diff?from=&to;=   | Return typed row and field differences.         |
| GET    | /api/projects/{id}/checkout/{commit} | Read a historical snapshot.                     |
| POST   | /api/projects/{id}/proofs            | Generate a proof for a root pair.               |
| POST   | /api/proofs/verify                   | Verify without loading the project dataset.     |
| GET    | /api/projects/{id}/stats             | Return storage, sharing and tree metrics.       |

## Definitions both partners must use consistently

- Version root: the hash identifying the complete structured state.
- Commit ID: the hash binding the version root, parent commit and metadata.
- Chunk reuse: bytes or chunks already present and referenced by a new version.
- Added/removed/updated: state differences by primary key, not merely recorded operations.
- Checkout: reading a historical state; it does not mutate the active branch by itself.
- Proof verification: recomputing trusted roots from supplied evidence, not trusting the server response.

## 10 RESEARCH EVALUATION

# How the metrics must be collected

## Research questions

| RQ  | Question                                                                                                               |
|-----|------------------------------------------------------------------------------------------------------------------------|
| RQ1 | How do fixed hash routing and ordered content-defined chunking trade update cost, range access and structural sharing? |
| RQ2 | How do commit and diff costs change with dataset size, change rate, history distance and update locality?              |
| RQ3 | How much time and data does independent diff verification require compared with transferring full snapshots?           |
| RQ4 | How does Chronos compare with Dolt on equivalent externally visible versioning operations?                             |

## Systems and workloads

| Axis             | Values / controls                                                                 |
|------------------|-----------------------------------------------------------------------------------|
| Systems          | Full snapshots; operation log; Python trie; Python/Go Prolly; pinned Dolt version |
| Rows             | 1K, 10K, 100K; 1M only if all systems complete reliably                           |
| Changes          | 1, 10, 100 and 1 percent of rows                                                  |
| History distance | Adjacent, 10 commits apart and 100 commits apart                                  |
| Distribution     | Random, clustered and append-heavy updates                                        |
| Record shape     | Fixed primary key; controlled small and medium payloads                           |
| Trials           | Fixed seed; warm-ups; at least 5-10 measured trials; same machine                 |

## Metrics and reporting

| Externally comparable                                                                    | Chronos-internal / algorithmic                                                                | Proof-specific                                                                     |
|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Initial load; incremental commit; diff; checkout; range scan; total storage; peak memory | New/reused chunks; bytes written; nodes read; tree depth; leaf occupancy; write amplification | Proof generation; verification; proof bytes; full-snapshot ratio; tamper detection |

Report medians and spread, exact versions, hardware, operating system, dataset seed and cache state. Never compare Chronos node counts with Dolt internal work unless Dolt exposes an equivalent measurement. Separate algorithmic conclusions from end-to-end system conclusions.

11 DELIVERY SCHEDULE

# Seven-day critical path

| Day | Backend / research                                        | Frontend / evaluation                   | Exit criterion                 |
|-----|-----------------------------------------------------------|-----------------------------------------|--------------------------------|
| 1   | Reconcile repository; freeze formats; pass Go Prolly gate | Wireframes; API types; demo dataset     | Go decision made; no ambiguity |
| 2   | Local mutation, persistent chunks, get/range scan         | Project/import shell; API client        | Restart-safe data path         |
| 3   | Commits, branches, log, checkout, diff, initial proofs    | Data workspace and history              | Complete backend workflow      |
| 4   | Proof hardening; HTTP integration tests                   | Diff, checkout, proof and stats screens | End-to-end frontend demo       |
| 5   | Pinned Dolt adapter; full experiment run                  | Evaluation charts; UX fixes             | Raw CSV results frozen         |
| 6   | Analyze results; address correctness defects              | Polish, paper figures and slides        | Release candidate              |
| 7   | Fresh setup and regression run                            | Rehearse; deploy; record backup demo    | Final artifacts submitted      |

## Suggested two-person ownership

| Owner                               | Primary responsibilities                                                                  | Daily handoff                                                  |
|-------------------------------------|-------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| Partner A - storage and backend     | Go tree, chunk store, commits, diff/proofs, API and correctness tests                     | Runnable binary, API contract update and test status           |
| Partner B - frontend and evaluation | React UX, dataset fixtures, Dolt adapter, experiment execution, charts and paper assembly | Frontend build, raw-results update and open integration issues |
| Both                                | Architecture decisions, claim review, demo rehearsal and final QA                         | 15-minute checkpoint with blockers and next-day exit criterion |

DAILY DISCIPLINE

**Integrate at least once per day. Keep the main branch green. A feature is not done until it has a correctness test and is visible through the shared API or benchmark harness.**

12 RISKS AND DECISION RULES

# Protect the deadline

| Risk                          | Early signal                                       | Mandatory response                                                    |
|-------------------------------|----------------------------------------------------|-----------------------------------------------------------------------|
| Prolly cursor complexity      | Boundary propagation fails random sequences        | Enforce Day-1 gate; switch platform backend to Python fallback.       |
| Repository divergence         | Remote changes and local edits overlap             | Create a safety branch/patch; reconcile before feature work.          |
| Persistence consumes schedule | Recovery semantics remain unstable on Day 3        | Use simple append-only chunks plus atomic refs; defer WAL.            |
| Frontend-backend mismatch     | UI waits for changing response shapes              | Freeze OpenAPI-like JSON examples on Day 1.                           |
| Unfair Dolt comparison        | Different workloads, startup inclusion or payloads | Use shared fixtures and state all unavoidable differences.            |
| Benchmark noise               | Large run-to-run variance                          | Pin load, seed and versions; report median plus spread.               |
| Proof incompleteness          | Valid-looking proof can omit a changed subtree     | Define completeness before implementation; include adversarial tests. |
| Scope expansion               | SQL, merge, auth or replication enters backlog     | Move it to future work immediately.                                   |

## Stop/go checkpoints

| Checkpoint | Continue only if                                  | Otherwise                                  |
|------------|---------------------------------------------------|--------------------------------------------|
| End Day 1  | Go Prolly correctness gate passes                 | Use Python engine; keep Go experimental    |
| End Day 3  | Persisted commit/diff/checkout works through API  | Cut branching before cutting core history  |
| End Day 4  | Frontend demonstrates the complete vertical slice | Stop feature work; integrate and polish    |
| End Day 5  | Raw benchmark data is reproducible                | Reduce matrix, never invent or cherry-pick |
| End Day 6  | Release candidate passes tests                    | Freeze scope; fix only blocking defects    |

SUBMISSION QUALITY RULE

A smaller system that is correct, reproducible, persistent, visually coherent, and honestly evaluated is stronger than a wider system containing unfinished database features.

## 13 DEFINITION OF DONE

# Final acceptance and demonstration checklist

## Engineering acceptance

| Check                                                                    | Pass |
|--------------------------------------------------------------------------|------|
| Fresh clone/build starts backend and frontend using documented commands  | [ ]  |
| Initial CSV import and commit completes with deterministic root          | [ ]  |
| Incremental update touches only a local region and ancestor path         | [ ]  |
| Old versions remain immutable and checkout correctly                     | [ ]  |
| Restart restores commits, refs and readable chunks                       | [ ]  |
| Diff reports additions, removals and field-level modifications correctly | [ ]  |
| Valid proof verifies; value/hash/path/root tampering fails               | [ ]  |
| Frontend completes the six-step user workflow without terminal use       | [ ]  |
| All automated tests pass                                                 | [ ]  |

## Research acceptance

| Check                                                                             | Pass |
|-----------------------------------------------------------------------------------|------|
| Benchmark command recreates raw CSV results from fixed fixtures                   | [ ]  |
| Dolt version and environment are pinned and recorded                              | [ ]  |
| Charts are generated from raw files, not manually transcribed                     | [ ]  |
| Paper separates algorithmic metrics from end-to-end metrics                       | [ ]  |
| Every conclusion has a result, limitation, or citation                            | [ ]  |
| Threats to validity disclose Python/Go, SQL overhead, cache and scale differences | [ ]  |

## Five-minute final demo script

| Time      | Action                                     | Message                                                          |
|-----------|--------------------------------------------|------------------------------------------------------------------|
| 0:00-0:40 | Open project and import CSV                | Structured data becomes one deterministic version root.          |
| 0:40-1:20 | Edit three rows and commit                 | Only affected chunks and ancestors are rewritten.                |
| 1:20-2:15 | Compare commits                            | Hash equality skips unchanged regions; diff is state-based.      |
| 2:15-2:50 | Checkout the original commit               | Historical state is immutable and immediately addressable.       |
| 2:50-3:45 | Generate, verify, then tamper with a proof | Roots authenticate the diff without both full datasets.          |
| 3:45-4:35 | Show evaluation charts                     | Results quantify update, diff, storage and proof trade-offs.     |
| 4:35-5:00 | State limitations and future work          | Chronos is a focused research datastore, not a Dolt replacement. |

14 PAPER FRAMING AND SOURCES

# The narrative to carry into the report

## Recommended title

*Chronos: Verifiable Versioned Structured Data with Content-Addressed Prolly Trees*

## Recommended paper structure

| Section                   | Purpose                                                         |
|---------------------------|-----------------------------------------------------------------|
| 1. Introduction           | Problem, motivation, gap, contributions and result summary.     |
| 2. Background             | Git/Merkle DAGs, content addressing, Prolly Trees and Dolt.     |
| 3. Chronos design         | Chunks, roots, commits, persistence, mutation and diff.         |
| 4. Verifiable diff proofs | Threat model, construction, verifier and completeness.          |
| 5. Implementation         | Go system, frontend, Python oracle and limitations.             |
| 6. Evaluation             | Research questions, workloads, metrics and results.             |
| 7. Discussion             | Trade-offs, threats to validity and Dolt comparison boundaries. |
| 8. Conclusion             | What was demonstrated and what remains future work.             |

## Core sources for background and related work

- DoltHub - How Dolt Stores Table Data: content addressing, structural sharing and recursive diff.
- DoltHub - Prolly Trees: history independence, chunk modification and fast diff.
- DoltHub - Efficient Diff on Prolly Trees: row-level tree-walk mechanics.
- DoltHub - Three-way Merge in a SQL Database: Dolt merge behavior and conflicts.
- DoltHub - Session-Aware GC: append-only chunk store and lifecycle.
- Dolt documentation - Benchmarks: official correctness and performance benchmark categories.

PARTNER ALIGNMENT QUESTION

**Before coding begins, both partners should agree to this sentence: the goal is a measured, verifiable versioned datastore - not a rushed miniature SQL database.**